

SQL

Def **Structured Query Language (SQL)** lets you access and manipulate databases. The standard language for manipulating databases

SQL is a standard of the American National Standards Institute

Def **Query In CS**, a query is a request directed to a database or information system to retrieve specific information

Def **Database** A **database** is an electronically stored, systematic collection of data (for example it can include words, numbers, images, videos, and other types of files).

Functions of SQL:

- SQL can execute queries against a database
- SQL can retrieve data from a database
- SQL can insert records in a database
- SQL can update records in a database
- SQL can delete records from a database
- SQL can create new databases
- SQL can create new tables in a database
- SQL can create stored procedures in a database
- SQL can create views in a database
- SQL can set permissions on tables, procedures, and views

DEFINITIONS

Def **Server** = Something that is meant to be communicated with over networks. Servers can communicate with another computer program and/or a database. A server is a computer or a system that provides resources, services, or data to other devices (clients) over a network.

- Servers use **database management systems (DBMS)** via a network.

Databases are Excel spreadsheets that can be tied together through keys. (An Excel spreadsheet on steroids). Databases are buckets of tables.

If **using SQL on a website** you will need a **RDBMS**

Def **RDBMS** = Relational Database Management System which stores database objects like tables.

- Ex: AWS has Amazon Relational Database Service (RDS)

Def **Tables** = A collection of related data entries (made of columns and rows)

Def **Fields** = Smaller entries within columns. Fields maintain specific information about every record (aka row) in a table

Def **Records** = Also known as a row is a individual entry that exists in a table (horizontal entity)

Def **Columns** = contains all necessary information associated with a specific field (vertical entity)

What you need for website: _____

1. A RDBMS database (MS Access, SQL Server, MySQL, Amazon RDS, Oracle Database)
2. Using some server-side scripting language (PHP or ASP)
3. Using SQL to get the data you want
4. Using HTML / CSS to style pages

Using SQLServer (Microsoft) _____

Lots of time will be concentrated under the Databases tab.

- Within your specific database, that is where you will spend 90% of your time
"New Query"

1st = Download data : Store the scripts where you want in your file system

2nd = Create your database

Go to database → Right click → New Database and name the database

3rd = Click create objects (**Make sure you are NOT running against master instead run against the database that you create**) Open script / data : Then create your objects first

- SQL Tutorials SQL-Server-Sample-Database has a BikeStores Database with three parts: "create objects," "drop all objects," and "load data."

4th = Execute : Should complete successfully. Then when you go to your object explorer make sure that the tables are created

5th = Refresh the Object Explorer

6th = Then load in your data → To check that your data is loaded into the database, go to Tables and then click "Show top 1000 rows" ... The data should be loaded in

Here is the step by step tutorial link for the six steps above:

<https://www.sqlservertutorial.net/getting-started/load-sample-database/>

Creating a table _____

- You can create a table using a script or using the GUI for the SQL Server Management. So start with "New Query" which is creating the new query.
- You are going to have columns and rows.
- Use a schema. A schema is a way to logically group data.
- Things that go inside the command when entering the command within script:
 - You are going to have a PRIMARY key in every single file.

PRIMARY KEY **uniquely identifies each record in a table**. Primary keys must contain UNIQUE values

- VARCHAR a variable character (use if storing names (because people's names vary in length)). A data type
- CHAR a fixed character (use if you are storing a code or a hash (something that will not change)). A data type

- Constraints → means you can't do certain things with your data
- NOT NULL means that there must be an entry when you are collecting the data
- UNIQUE means that the entry collected must be unique
- FOREIGN Keys help us tie tables together

A FOREIGN KEY is a field (or collection of fields) in one table, that **refers to** the PRIMARY KEY in another table. The table with the foreign key is called the child table, and the table with the primary key is called the referenced or parent table.

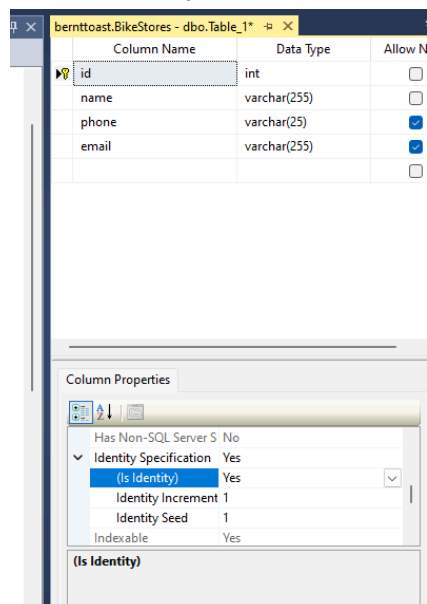
USE BikeStores;

```
CREATE TABLE production.trainer (
    id INT IDENTITY(1, 1) PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    char_id INT NOT NULL,
    FOREIGN KEY (char_id)
        REFERENCES production.characters(id)
        ON DELETE CASCADE ON UPDATE CASCADE
);
```

In SQL you can set a primary key by going to the column, right click, select “set primary key” - will then automatically uncheck so it cannot be null.

Then you go in and select what cannot and can be null.

Set the identity:



This GUI entered table is under the Bike Stores database → “production.primary-and-schema”

INSERT Statement: INSERT INTO + {schema.table name}

To insert into a table using the script version:

```
INSERT INTO product.poke                                #where product.poke = schema.tablename
VALUES('Charmander', '818-227-6224', 'charmander@email.com') #the values
```

```
INSERT INTO production.characters (name, phone, email)
```

```
VALUES ('Charmander', '818-227-6224', 'charmander@email.com');
```

OR you can use the GUI and right click and then edit / enter the values

Note: Never use double quotes in SQLserver. Only use single quotes: ' '

UPDATE Statement: UPDATE + {schema.table_name}

- Easiest way is to do so in the GUI. But the GUI only lets you edit the top 200 rows. Also not "professional"
- So instead we write the Query:

Example:

```
UPDATE production.pokemon
```

```
SET name = 'Pokeballpine', phone = '777-777-7777'
```

```
WHERE id = 2
```

→ Then Run the Query (tested by changing bike_brands id 2 to

'Pokepine' and it worked.

Next... start new:

<https://www.youtube.com/watch?v=IneUOPTjSRc&list=PL82C6-O4XrHfZoh2ZH7-HCPyh9oHeYPnz&index=7>

Querying data from SQL Server

GENERAL SYNTAX

Select, update, delete, insert, where

- SELECT = extracts data from a database (selects the data from a database). Shows you how to query data against a single table
 - "Select Top 1000 rows" → Then you will write your queries
 - SELECT DISTINCT: Statement used to return only distinct (different) values
 - SELECT COUNT(DISTINCT Country) FROM Customers; // to return number of countries [For Microsoft: `SELECT Count(*) AS DistinctCountries FROM (SELECT DISTINCT Country FROM Customers);`]
 - SELECT DISTINCT = Useful for removing duplicates
- UPDATE = updates the data in a database
- DELETE = deletes the data from a database ALSO KNOWN AS DROPPING
- INSERT INTO = inserts new data into a database
- CREATE DATABASE = creates a new database
- ALTER DATABASE = modifies a database
- CREATE TABLE = creates a new table

- ALTER TABLE = modifies a table
- DROP TABLE = deletes a table
- CREATE INDEX = creates an index (search key)
- DROP INDEX = deletes an index
- WHERE = Used to extract only the records that fulfill a specified condition. It filters the rows in the output of a query based on one or more conditions
 - Example: `SELECT * FROM Customers WHERE Country='Mexico';`
 - Example: `SELECT * FROM Customers WHERE CustomerID=1;`
 - You can use operators (< , > , =, BETWEEN, IN, <>, etc.) with the WHERE clause
- ORDER BY = Sorts the result based on values in a specified list of columns (To sort the data)
 - Example:


```
SELECT * FROM Products
ORDER BY Price
```
- OFFSET FETCH = Limits the number of rows returned by a query
- SELECT TOP = Limits the number of rows or percentage of rows returned in a query's result set
- DISTINCT = Selects distinct values in one or more columns of a table
- Subquery = queries nested within another statement like SELECT, INSERT, UPDATE, or DELETE

EXAMPLES

Example: To select all records from a table called 'Customers' use:

```
SELECT * FROM Customers;
```

Continue here: <https://www.sqlservertutorial.net/getting-started/load-sample-database/>
https://www.w3schools.com/sql/sql_orderby.asp

Sources: W3Schools,

<https://www.youtube.com/playlist?list=PL82C6-O4XrHfZoh2ZH7-HCPyh9oHeYPnz>